Curso de Especialización en IA y BigData
Sistemas de Aprendizaje Automático

Laboratorio 2: Almacenamiento y recuperación de datos en Python

1. Almacenamiento de los *datasets*

En el contexto del **Machine Learning**, un **dataset** es el conjunto de datos que se utiliza para entrenar, validar y evaluar a un modelo de aprendizaje automático. Es una **colección organizada de ejemplos** que representan el problema que queremos resolver.

Los datos que contiene un *dataset* se pueden guardar en almacenamiento secundario o persistente, para su conservación a largo plazo, o bien en almacenamiento primario o transitorio, para su procesamiento por parte de programas en ejecución.

Como medio de almacenamiento persistente se puede utilizar una base de datos relacional. Pero es muy frecuente utilizar ficheros de diversos tipos.

Los más habituales son:

- **Ficheros CSV** (*Comma-Separated Values*). Tienen muchas variantes. La característica común a todos ellos es que son ficheros de texto, y en cada línea están los valores de los atributos para una observación o instancia, separados por un carácter separador de campos. El separador de campos más habitual es una coma (y de ahí su nombre). Pero podría ser también un tabulador, un espacio, o cualquier otro. Cuando solo hay datos numéricos y se utiliza un punto como separador de decimales, se puede utilizar una coma como se parador de campos. Pero si se usa una coma como separador de decimales, habría que utilizar otro carácter como separador de campos, por ejemplo punto y coma. Si el valor de algún atributo se representa mediante cadenas de caracteres, entonces el separador de campos debería ser un carácter que no aparezca dentro de los valores para los atributos. Una buena elección es el tabulador, porque no suele aparecer dentro de los textos. Si entrecomillamos los valores, en general, no habría inconveniente en utilizar cualquier carácter como separador de campos (excepto las propias comillas, claro está). Solo quedaría el problema de que las propias comillas aparezcan dentro de los valores de los atributos. Entonces habría que utilizar un mecanismo de *escaping* para que no se interpreten como delimitadores de campos, sino como comillas dentro de una cadena de caracteres. El más habitual para formato CSV es el redoblamiento (doble comillas).

  Normalmente, **en la primera línea** vienen los nombres de los atributos, y en cada una de las líneas siguientes, los valores para los atributos de cada observación o instancia. Pero

esto no es necesariamente así siempre. A veces se dan solo los valores de los campos, y los nombres de los campos están documentados en otra parte.

- **Ficheros ARFF** (Attribute-Relation File Format). Es un formato usado por Weka, del que se pueden encontrar información en

  https://waikato.github.io/weka-wiki/formats_and_processing/arff_stable

  Sigue siendo un format sencillo pero menos rudimentario que CSV. Permite especificar el tipo de los distintos atributos, e incluir comentarios y explicaciones.

Valores nulos: se representan de distintas formas según el medio de almacenamiento:

- **Fichero CSV**: Se deja vacío el campo, con lo que aparecen dos separadores seguidos. Si los valores nulos están al final de la línea, se puede terminar la línea después del último atributo con valor no nulo, es decir, se pueden omitir las comas o separadores de campos del final de la línea.
- **Fichero ARFF:** Los **valores nulos** se representan con un **signo de interrogación (?)**. Para el ***escaping*** se utiliza un mecanismo análogo al de los lenguajes C y Java, es decir, añadir un carácter \ (*backs- lash* o barra invertida) para evitar que un carácter se interprete como un carácter especial. El carácter \ se representaría como \\.

Existen varios **sitios web** donde se pueden encontrar multitud de *datasets* disponibles en ficheros para su descarga. Además de los propios ficheros, se puede encontrar mucha documentación y análisis pormenorizados acerca de los *datasets*, y explicaciones detalladas acerca de la aplicación de técnicas de *Machine Learning* sobre ellos.

- UCI Machine Learning Repository. https://archive.ics.uci.edu/datasets
- Kaggle. https://www.kaggle.com/datasets
- Universidad de Waikato. https://github.com/Waikato/weka-3.8/tree/master/wekadocs/data

Actividad 1.1

a) Acceder al repositorio de la UCI para ML, *UCI Machine Learning Repository* (https://archive.ics.uci.edu/datasets), donde hay disponibles muchos y diversos *datasets* en formato CSV. Localiza y descarga los siguientes dataset:

- Iris.
- Heart Disease.
- Wine Quality.
- Wine.
- Auto MPG.
- Mushroom.

Describe el archive IRIS y WINE QUALITY indicando:

- Que información contienen
- Nº de columnas que incluyen?
- Nº de registros que incluye?

Accede tambien a Kaggle (https://www.kaggle.com/datasets), y a los siguientes *datasets:*

- Boston Housing.
- Titanic - Machine Learning from Disaster

Describe su contenido indicando:

- Que información contienen?
- Nº de columnas que incluyen?
- Nº de registros que incluye?

b) Investiga la lista de dataset que encontramos en https://github.com/Waikato/weka-3.8/tree/master/wekadocs/data. ¿En que format se encuentran?

Actividad 1.2

Para localizar dataset en format ARFF puedes navegar en las siguientes web. Comprueba que están activas y accesibles:

- https://waikato.github.io/weka-wiki/datasets/
- https://github.com/renatopp/arff-datasets
- https://www.cs.ubc.ca/labs/algorithms/Projects/autoweka/datasets/
- https://www.uco.es/kdis/mllresources/
- https://storm.cis.fordham.edu/~gweiss/data-mining/datasets.html
- https://github.com/deric/clustering-benchmark

2. *Machine Learning* con Python

Python es el lenguaje que desde hace tiempo se ha impuesto en el campo del *Machine Learning* y en general en el campo del cálculo científico.

Algunos motivos para ello son:

- Es un lenguaje interpretado que permite introducir expresiones y evaluarlas de forma interactive y generar gráficos inmediatamente.
- Es un lenguaje eficiente. Gran parte de las bibliotecas que llevan a cabo procesos intensivos en cálculo están implementadas en otros lenguajes de más bajo nivel y más eficientes, principalmente C/C++, y también Fortran.
- Permite operaciones vectoriales, es decir, que se realizan sobre secuencias de valores. Esta es una característica propia de lenguajes funcionales, muy conveniente para *Machine Learning*.
- Es un lenguaje extremadamente flexible y versátil. Es multiparadigma: tiene características de lenguajes imperativos, orientados a objetos y, como se ha apuntado ya, funcionales.

Las bibliotecas más habitualmente utilizadas para Machine Learning son las siguientes:

NumPy	Cálculo científico	NumPy
Pandas	Análisis y manipulación de *datasets*	pandas

Matplotlib	Gráficos	
Seaborn	Visualización de datos estadísticos	
Scikit-Learn	Machine Learning	
TensorFlow	Machine Learning, especialmente para redes neuronales y *Deep Learning*	
Keras	Redes neuronales. API de alto nivel que suele usar TensorFlow como *backend*.	

Existe una convención universalmente aceptada para los alias con los que se importan estas bibliotecas en Python, que se muestra en la siguiente tabla. En algunos de los programas de ejemplo es posible que se omitan algunas de estas sentencias al principio, por brevedad, y se utilicen directamente los alias np, pd, plt, etc.

Paquete	Sentencia para importación	
`numpy`	`import numpy as np`	
`pandas`	`import pandas as pd`	
`matplotlib.pyplot`	`import matplotlib.pyplot as plt`	
`seaborn`	`import seaborn as sns`	
`tensorflow`	`import tensorflow as tf`	
`keras`	`from tensorflow import keras`	Se puede usar la implementación incluida en TensorFlow, o independientemente. En principio da igual.
	`import keras`	

Llegado este momento, se entiende que dispones de un entorno de ejecución y de desarrollo de Python instalado en una máquina local.

3. Carga de *datasets* desde ficheros con Python

En python se pueden cargar dataset de diversas formas. Las más habituales son cargar un d*atasets* **desde ficheros en formatos CSV y/o ARFF**. Tambien podemos hacerlo a traves de una **URL** o incluso **desde una libreria**. El formato CSV es el que utilizaremos con mayor frecuentemente.

Los datos del archivo CSV o ARFF se vuelcan entonces en un ***DataFrame de pandas** para poder manipularlos en el programa.

Un DataFrame es una estructura de datos bidimensional que ofrece la librería **pandas en Python. Puedes imaginarlo como una hoja de cálculo de Excel o una tabla en una base de datos, con filas y columnas. Cada columna puede tener un tipo de dato distinto (números, texto, fechas, booleanos, etc.), y cada fila representa un registro o instancia de datos.*

Como primer ejemplo se muestra a continuación la carga del *dataset* de las flores Iris. Este contiene datos de especímenes de flores de distintas especies del género Iris. Se puede descargar desde los siguientes URLs.

- https://www.kaggle.com/datasets/uciml/iris/download. En formato CSV.
- https://github.com/Waikato/weka-3.8/tree/master/wekadocs/data. En formato ARFF.

Descargalos a una carpeta de tu ordenador para tenerlos localizados localmente.

IMPORTANTE: ***Si trabajas en local, el archive con el dataset debe situarse en la carpeta donde se situe el archive con el programa.***

3.1. Carga de datos en formato ARFF

Para la carga de datos desde un fichero, el formato ARFF es en principio más sencilla y directa que desde un fichero CSV, porque no hay que tener en cuenta tantas variantes como para un fichero CSV (como por ejemplo separador de campos y de decimales). Si el fichero es correcto, debería cargarse sin más.

El siguiente código carga el *dataset*, desde el fichero iris.arff, en un DataFrame de pandas.

```
from scipy.io import arff
import pandas as pd

valores, metad = arff.loadarff("iris.arff")

df1 = pd.DataFrame(valores)
```

IMPORTANTE: En Python, una función o método puede devolver más de un valor.

La llamada a ***loadarff*** devuelve en ***valores*** un *array* de tuplas, cada una con una fila del *dataset*, y en ***metad*** metadatos que describen el tipo de cada atributo. El ***DataFrame*** se crea a partir del contenido de la variable ***valores***.

```
[1] from scipy.io import arff
    import pandas as pd

    valores, metad = arff.loadarff("iris.arff")

    df = pd.DataFrame(valores)
```

```
df
```

	sepallength	sepalwidth	petallength	petalwidth	class
0	5.1	3.5	1.4	0.2	b'Iris-setosa'
1	4.9	3.0	1.4	0.2	b'Iris-setosa'
2	4.7	3.2	1.3	0.2	b'Iris-setosa'
3	4.6	3.1	1.5	0.2	b'Iris-setosa'
4	5.0	3.6	1.4	0.2	b'Iris-setosa'
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	b'Iris-virginica'

figura 3.1: Contenido del Dataframe df

En la figura 1, lacolumna ***class*** muestra los valores de la clase entre comillas y precedidos de b (por ejemplo: b'Iris-setosa'). Estas son cadenas de caracteres codificadas como bytes en formato UTF-8. En la práctica es necesario contar con valores de texto normal. Podemos hacer la conversion con la siguiente linea de código Python. Esto es necesario para que funcionen los algoritmos de Machine Learning de Scikit-Learn.

```
df1['class'] = df['class'].str.decode("utf-8")
```

3.2. Carga de datos en formato CSV

La carga de datos en formato CSV es muy sencilla. La única dificultad que puede haber es la gran cantidad de variantes que puede tener este formato. Hay que echarle un vistazo al fichero. El archive en formato CSV para las flores Iris comoenza como se muestra a continuación:

```
Id,SepalLengthCm,SepalWidthCm,PetalLengthCm,PetalWidthCm,Species
1,5.1,3.5,1.4,0.2,Iris-setosa
2,4.9,3.0,1.4,0.2,Iris-setosa
3,4.7,3.2,1.3,0.2,Iris-setosa
4,4.6,3.1,1.5,0.2,Iris-setosa
5,5.0,3.6,1.4,0.2,Iris-setosa
6,5.4,3.9,1.7,0.4,Iris-setosa
7,4.6,3.4,1.4,0.3,Iris-setosa
8,5.0,3.4,1.5,0.2,Iris-setosa
9,4.4,2.9,1.4,0.2,Iris-setosa
10,4.9,3.1,1.5,0.1,Iris-setosa
11,5.4,3.7,1.5,0.2,Iris-setosa
```

figura 3.2: Contenidos de fichero Iris.csv

El fichero mantiene el format más habitual: el ***separador de campos es la coma***, y el ***separador de decimales para los números es el punto***.

El siguiente código carga el *dataset* desde el fichero Iris.csv en un DataFrame de pandas. La función “supone” que el archivo sigue las opciones por defecto anteriores.

```
import pandas as pd

df2 = pd.read_csv("Iris.csv")

df2
```

Una vez cargados los datos del fichero en el DataFrame, se puede verificar de manera sencilla su contenido:

```
[4] import pandas as pd
    df = pd.read_csv("Iris.csv")
```

df

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...	...
145	146	6.7	3.0	5.2	2.3	Iris-virginica
146	147	6.3	2.5	5.0	1.9	Iris-virginica
147	148	6.5	3.0	5.2	2.0	Iris-virginica
148	149	6.2	3.4	5.4	2.3	Iris-virginica

figura 3.2: Contenidos del Dataframe una vez cargado el fichero CSV

No siempre la carga de datos a partir de un fichero en formato CSV será tan directa como con este ejemplo, en el que basta con indicar el nombre del fichero.

La función read_csv de pandas tiene muchos parámetros para poder cargar ficheros en formato CSV con diversas opciones. Algunos de los más importantes son los de la tabla siguiente. Se puede encontrar toda la información al respecto en:

https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html

Nombre	Tipo	
sep	*str*	Separador de campos. Valor por defecto: `','` Si no se especifica ninguno, normalmente se podrá detectar el separador automática - mente. El separador puede ser no solo un carácter, sino también una cadena de caracte - res e incluso una expresión regular. Si el separador de caracteres es la coma, que es lo más habitual, no hará falta especificarlo. En otro caso, es conveniente especificarlo.
delimiter	*str*	Es un alias para el anterior parámetro, `sep`.
delim_whitespace	*bool*	Equivalente a sep=`'\s+'`. Con esta opción, se reconocen como separadores secuencias de espacios en blanco. No debería especificarse ningún valor para el parámetro `sep`. Si se especifica esta opción.
decimal	*str*	Separador de decimales. Valor por defecto: `'.'`
thousands	*str*	Separador de miles.
names	*array-like*	Nombres de columnas. Es útil cuando en la primera línea no están los nombres de las columnas, o cuando están pero se quieren utilizar otros nombres. En este último caso, hay que utilizar la opción `header=0`.
lineterminator	*str*	Separador de líneas. Por defecto es un retorno de carro.
skipinitialspace	*bool*	Ignorar espacios después del delimitador o separador de campos. Valor por defecto: `True`.
na_values	*scalar*, *str*, *list-like*, o *dict*	Cadenas de caracteres que hay que interpretar como valores nulos (missing), adem ás de las que se reconocen por defecto como tales. Las cadenas que se reconocen por de- fecto como valores nulos son, según la documentación: `''`, `'#N/A'`, `'#N/A N/A'`, `'#NA'`, `'-1.#IND'`, `'-1.#QNAN'`, `'-NaN'`, `'-nan'`, `'1.#IND'`, `'1.#QNAN'`, `'<NA>'`, `'N/A'`, `'NA'`, `'NULL'`, `'NaN'`, `'n/a'`, `'nan'`, `'null'`
keep_default_na	*bool*	Interpretar como valores nulos los valores por defecto interpretados como tales. Valor por defecto: `True`.
skiprows	*int* o *list-like*	Número de líneas que hay que ignorar al principio, o bien lista con números de línea que hay que ignorar.
skipfooter	*int*	Número de líneas que hay que ignorar al final del fichero.
nrows	*int*	Número de líneas que hay que leer. Útil para leer solo parte de un fichero muy largo.

Tabla 2: Parámetros de la función `read_csv` de `pandas`.

```
import pandas as pd

df3 = pd.read_csv("Iris.csv", sep=",")

df3
```

3.3. Carga de datos incluidos en una libreria determinada

En **Machine Learning** no siempre tienes que cargar un dataset desde un archivo CVS o ARFF, muchas veces se usan **datasets ya preparados** que vienen incluidos en librerías o repositorios públicos. Por ejemplo, la librería **Scikit-learn (Python) i**ncluye datasets clásicos de ejemplo:

- load_iris() → clasificación de flores.
- load_digits() → reconocimiento de dígitos escritos a mano.
- load_wine() → calidad de vinos.
- load_breast_cancer() → diagnóstico de cáncer.

Para preparar un DataFrame con los registros del dataset se pueden utilizar las siguientes funciones:

```
import pandas as pd
from sklearn.datasets import load_iris

# Cargar dataset
data = load_iris()

# Crear DataFrame con las características
df3 = pd.DataFrame(data.data, columns=data.feature_names)

# Añadir la columna resultado u objetivo
df3['target'] = data.target

# Por defecto muestra las primeras 5 filas. df.head(15) ostraría las 15 primeras
df3.head()
```

4. Consulta de datos de un *DataFrame* de pandas

En la siguiente tabla se muestran algunos atributos y métodos que serán útiles, en adelante, para consultar los contenidos del DataFrame. Esta es solo una pequeña parte de todo lo disponible en la clase. Se puede encontrar más información acerca de la clase ***pandas.DataFrame*** en la documentación de pandas, en

https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html.

Atributos	`columns`	Nombres de columnas del `DataFrame`. Estos son los atributos del *dataset*.
	`dtypes`	Tipos de datos de las columnas del `DataFrame`.
	`shape`	Dimensiones del `DataFrame`. Devuelve una tupla con dos elementos: el número de filas y el número de columnas.
	`values`	Valores contenidos en el `DataFrame`, en forma de *array* bidimensional.
Métodos	`head([n])`	Primeras n filas. Por defecto, n=5.
	`tail([n])`	Últimas n filas. Por defecto, n=5.
	`info()`	Es muy interesante porque muestra informaci ón adicional a la mostrada por `dtypes`, como por ejemplo el número de valores nulos (`Null`) en cada columna.

Cuadro 1.1: Atributos y métodos de la clase `DataFrame` de `pandas`

Actividad 1.4

Una vez obtenido un DataFrame a partir de cualquiera de los ficheros anteriores, en formato CSV o ARFF, añade al programa de prácticas instrucciones que hagan uso de los atributos y metodos anteriores. *(añade un comentario al programa indicando que las instrucciones corresponde a la actividad 1.4)*

5. Selección de columnas de un DataFrame

Se puede obtener un DataFrame solo con las columnas que interesan. Algunos ejemplos:

```
df4=df[["SepalLengthCm"]]
df5=df[["SepalLengthCm", "SepalWidthCm"]]
df4
df5
```

También se puede obtener un DataFrame con todas las columnas menos algunas. Por ejemplo:

```
df6=df.drop(["Id"], axis=1)
df7=df.drop(["Id", "Species"], axis=1) # axis=1 para eliminar columnas.
df6
df7
```

La ejecución del método ***drop*** no altera el DataFrame de origen sobre el que se ejecuta. El método devuelve un nuevo DataFrame al que se le elimina la columna o fila indicada. ***(axis=1 para eliminar columnas y axis=0 para filas)***

Si se desea eliminar la columna en el propio DataFrame de origen se puede utilizar la opción ***inplace = True.***

```
df6=df.drop(["Id"], axis=1, inplace=True)
df6
```